

Assignment 5: Pacman Protocol

Design and Specification

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

This session introduces the "Pacman Protocol Specification" assignment. We will review the context of the multi-player game, analyse the flaws in the current networking implementation, and define the requirements for designing a robust, secure, and language-independent protocol specification.

Lecture Overview

- 1 Introduction and Context
- 2 The Current Implementation
- 3 Specification Requirements
- 4 Submission and Marking

Introduction and Context

Introduction and Context

The Assignment

The Goal

Your task is **not** to write code. Instead, you must write a technical **Protocol Specification** for a multi-player networked Pacman game.

Why Specification?

Writing code is only half the job. In professional engineering, the ability to communicate *how* a system works—unambiguously and concisely—is critical for interoperability and maintenance.

The Assignment

The Goal

Your task is **not** to write code. Instead, you must write a technical **Protocol Specification** for a multi-player networked Pacman game.

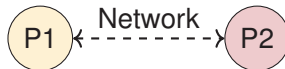
Why Specification?

Writing code is only half the job. In professional engineering, the ability to communicate *how* a system works—unambiguously and concisely—is critical for interoperability and maintenance.

The Game Scenario

We have extended the classic Pacman game to support two players.

- **Original Intent:** A sociable game played by two people in the same room.
- **Mechanism:** If your Pacman enters the tunnel, it disappears from your screen and appears on your opponent's screen.
- **Remote Mode:** Added during the pandemic to allow play across the internet, displaying a "mini-view" of the opponent's screen.



Game Terminology

To specify the protocol, we must agree on terminology regarding where objects are.

LOCAL: A game object currently on your own screen.

AWAY: Your Pacman when it has travelled to the remote screen.

REMOTE: A game object (like a ghost) on the remote screen that your **AWAY** Pacman interacts with.

FOREIGN: The opponent's Pacman when it is visiting your screen.

Implication

Your local model must track both the **Local Maze** and the **Remote Maze** to handle interactions efficiently.

Game Terminology

To specify the protocol, we must agree on terminology regarding where objects are.

LOCAL: A game object currently on your own screen.

AWAY: Your Pacman when it has travelled to the remote screen.

REMOTE: A game object (like a ghost) on the remote screen that your **AWAY** Pacman interacts with.

FOREIGN: The opponent's Pacman when it is visiting your screen.

Implication

Your local model must track both the **Local Maze** and the **Remote Maze** to handle interactions efficiently.

The Current Implementation

The Current Implementation

The Existing Protocol

The current alpha-release contains a working network protocol, but it is deeply flawed.

Origin Story

Implemented by an anonymous professor late at night after a pub visit. It is a "quick and dirty" solution.

Major Flaws

- 1 **Language Dependent:** Uses Python's 'pickle' module.
- 2 **Insecure:** 'pickle' is vulnerable to malicious input (arbitrary code execution).
- 3 **Ugly Encoding:** A mix of binary length headers, text types, and pickled payloads.
- 4 **Inefficient:** "Chatty" behaviour (e.g., sending four separate ghost updates per frame).

The Existing Protocol

The current alpha-release contains a working network protocol, but it is deeply flawed.

Origin Story

Implemented by an anonymous professor late at night after a pub visit. It is a "quick and dirty" solution.

Major Flaws

- 1 Language Dependent:** Uses Python's 'pickle' module.
- 2 Insecure:** 'pickle' is vulnerable to malicious input (arbitrary code execution).
- 3 Ugly Encoding:** A mix of binary length headers, text types, and pickled payloads.
- 4 Inefficient:** "Chatty" behaviour (e.g., sending four separate ghost updates per frame).

Why 'pickle' must go

- **Interoperability:** A protocol using 'pickle' can only be spoken by Python programs. A standard protocol should be implementable in C++, Java, Rust, etc.
- **Security:** Deserialising untrusted 'pickle' data is dangerous.

Your Challenge

Design a replacement protocol that is **clean**, **secure**, and **language-agnostic**.

Specification Requirements

Specification Requirements

Core Constraints

Your specification must adhere to the following strict constraints:

1. Encoding Format

- You must choose **Text-based** (e.g., ASCII commands) OR **Binary-based** encoding.
- Do **not** mix them without a compelling documented reason.
- **Forbidden:** You may NOT use JSON, XML, HTML, or 'pickle'. You must design the byte-level or character-level format yourself.

2. Transport Layer

- You may use **TCP**, **UDP**, or a combination of both.
- If using UDP, you must specify how to handle packet loss (e.g., retransmission or ignoring outdated state).

Core Constraints

Your specification must adhere to the following strict constraints:

1. Encoding Format

- You must choose **Text-based** (e.g., ASCII commands) OR **Binary-based** encoding.
- Do **not** mix them without a compelling documented reason.
- **Forbidden:** You may NOT use JSON, XML, HTML, or 'pickle'. You must design the byte-level or character-level format yourself.

2. Transport Layer

- You may use **TCP**, **UDP**, or a combination of both.
- If using UDP, you must specify how to handle packet loss (e.g., retransmission or ignoring outdated state).

Scope of Messages

You must specify how to encode information for all game events. The current protocol uses 12 message types:

- | | |
|-------------------------------------|------------------------------------|
| 1 Maze update (sent at start) | 7 Ghost update |
| 2 Pacman arrived (tunnel transit) | 8 Ghost was eaten |
| 3 Pacman left | 9 Foreign pacman ate ghost |
| 4 Pacman died | 10 Eat (food/powerpill) |
| 5 Pacman go home | 11 Score update |
| 6 Pacman update (position/velocity) | 12 Status update (Game Over, etc.) |

Note: Your new protocol does not need to map these 1:1, but it must cover all functionality.

Sanity Checking and Robustness

A key part of a specification is defining validity.

Defensive Design

Your specification must indicate how received values should be checked.

- **Example:** If a message claims "Ghost #5" moved, but there are only 4 ghosts, what should happen?
- **Example:** If a coordinate is outside the maze bounds?
- The specification must be unambiguous enough for another programmer to implement these checks.

Submission and Marking

Submission and Marking

Deliverables

Submission

- Submit a **PDF Document** via Moodle.
- This document is a technical specification, not a story.
- No code submission is required for this assignment.

Peer Marking

You will mark the specifications of your peers within your group. **Mental Model:** When marking, ask yourself: *"Could I implement the game networking using ONLY this document, without guessing?"*

Marking Criteria

You should optimise your specification for the following qualities:

Conciseness: Be specific. Do not "waffle". This is technical documentation, not literature.

Correctness: Will the protocol actually work? Are there logical flaws?

Unambiguity: Is there only one way to interpret the byte layout?

Completeness: Are edge cases (like Game Over or connection loss) handled?

Examples

You may use examples to clarify complex parts (non-normative), but they should not replace the formal definition.

Marking Criteria

You should optimise your specification for the following qualities:

Conciseness: Be specific. Do not "waffle". This is technical documentation, not literature.

Correctness: Will the protocol actually work? Are there logical flaws?

Unambiguity: Is there only one way to interpret the byte layout?

Completeness: Are edge cases (like Game Over or connection loss) handled?

Examples

You may use examples to clarify complex parts (non-normative), but they should not replace the formal definition.

Getting Started

- 1 **Run the Game:** Download the alpha code. Run the server and client (or relay server). Play it (use `-r` for remote mode).
- 2 **Inspect the Code:** Look at `pa_network.py` and `pa_model.py`.
- 3 **Debug:** Add `print()` statements to `check_for_messages()` to see what the current bad protocol is actually sending.
- 4 **Design:** Draft your message structures on paper before writing the spec.

Warning

The assignment document describes the game, but if there is a conflict between the document and the Python code behaviour, **the code is authoritative**.

Summary I

Key Takeaways

- **Task:** Design a replacement network protocol for Pacman.
- **Constraint:** No 'pickle', no JSON/XML. Consistent text or binary encoding.
- **Scope:** Handle all game states (Movement, Eating, Tunnels, Game Over).
- **Goal:** Produce a complete, unambiguous specification document.
- **Deadline:** Check Moodle.